

Atlas Architecture Design

Atlas Architecture Group

August 17, 2026

Contents

0.1	0. 规范性引用与文档治理	2
0.1.1	0.1 Normative Architecture References	2
0.1.2	0.2 规范优先级	2
0.2	1. 架构愿景与定位	2
0.3	2. 核心设计原则	3
0.3.1	2.1 Bootstrap ≠ GitOps	3
0.3.2	2.2 四阶控制权模型	4
0.3.3	2.3 External Root Anchor + Two-Level Reconciliation DAG	6
0.3.4	2.4 Directory Ownership ≠ Dependency Ordering	9
0.3.5	2.5 GitOps Reconciliation Authority	10
0.3.6	2.6 Artifacts ≠ Intent	10
0.3.7	2.7 Developer Cognitive Load Reduction	11
0.4	3. 标准目录与物理信任边界	12
0.4.1	3.1 __INLINE_CODE_0__	13
0.4.2	3.2 __INLINE_CODE_0__	14
0.5	4. Atlas 控制拓扑	14
0.6	5. 工程约束与防御机制	15
0.6.1	5.1 Dual Deletion Protection	15
0.6.2	5.2 Bootstrap Adoption Invariant	16
0.6.3	5.3 Zero-Plaintext Secret	17
0.6.4	5.4 Tier-0 Human Judgment	18
0.7	6. Unified Telemetry & Observability	18
0.7.1	6.1 Metrics	18
0.7.2	6.2 Collection	19
0.7.3	6.3 Logs	19
0.7.4	6.4 Traces	19
0.8	7. Disaster Recovery Boundaries	19
0.8.1	7.1 Disposable State	20
0.8.2	7.2 Must-Backup State	20
0.8.3	7.3 Application-consistent Recovery	20
0.9	8. Break-Glass Architecture Principle	21
0.9.1	Freeze	21
0.9.2	Recover	22
0.10	9. Architecture Invariants	22
0.10.1	INV-01 —Definition Authority	22
0.10.2	INV-02 —Bootstrap Termination	22
0.10.3	INV-03 —External Root	22
0.10.4	INV-04 —GitOps Authority	22
0.10.5	INV-05 —Runtime Authority	22

0.10.6 INV-06 —Two-Level DAG	22
0.10.7 INV-07 —Root Minimalism	22
0.10.8 INV-08 —Domain / Dependency Separation	22
0.10.9 INV-09 —Leaf Failure Isolation	23
0.10.10 INV-10 —Bounded Application Depth	23
0.10.11 INV-11 —Adoption Parity	23
0.10.12 INV-12 —Control/Data Failure Isolation	23
0.10.13 INV-13 —Zero Plaintext Secret	23
0.10.14 INV-14 —Offline Artifact Availability	23
0.10.15 INV-15 —Stateful Recovery Semantics	23
0.10.16 INV-16 —Tier-0 Human Judgment	23
0.11 10. Conformance Model	23
0.12 11. Architecture Freeze Statement	24

Version: v1.0.2

Status: Frozen Production Baseline (Topology Conformance Amendment)

Owner: snkio027

Scope: Kubernetes Runtime, Argo CD GitOps Control Plane, Developer Platform Foundation, Streaming/Lakehouse Infrastructure

0.1 0. 规范性引用与文档治理

Atlas 采用分层架构文档体系。不同层级的设计文档分别拥有不同的规范权威，以避免平台级原则、领域级实现语义与具体工程实现相互覆盖。

0.1.1 0.1 Normative Architecture References

1. **Atlas Architecture Design v1.0.2 (本文档)** 定义 Atlas 的平台级架构原则、信任边界、控制权模型、系统不变量、容灾边界与领域分层。
1. **Atlas GitOps Control Plane Design v1.0.3** 定义 GitOps 控制域内的具体执行语义，包括 External Root Anchor、AppProject 自举链、Two-Level Reconciliation DAG、Sync Wave、Application Health Gate、删除保护与 Break-Glass Recovery。
1. **AI-Native Platform Control Plane Standard v1.4** 定义 AI Agent 的交互权限、Human Judgment Gate、鉴权、审计与安全兜底模型。

0.1.2 0.2 规范优先级

Atlas 明确区分：

- **Global Architecture Invariant**：由本文档定义，领域设计不得违反。
- **Domain Execution Semantics**：由对应领域设计定义，在本文档授权范围内拥有更高的执行细节权威。
- **Implementation Detail**：由代码、Manifest、Helm Values、Policy 等具体工程资产实现。

因此：

Domain Design MAY refine Architecture semantics, but MUST NOT override Architecture invariants.

当领域文档与本文档中的非规范性示例冲突时，以领域文档为准；当领域文档与本文档定义的平台级不变量冲突时，以本文档为准。

本文中的 **MUST / MUST NOT / SHOULD / SHOULD NOT / MAY** 具有规范性约束意义。

0.2 1. 架构愿景与定位

Atlas 是面向云原生数据基础设施与流处理场景的内部开发者平台（Internal Developer Platform, IDP）参考架构。

其首要目标是在：

- macOS Apple Silicon
- OrbStack Docker Runtime

- Kubernetes
- Argo CD
- Streaming / Lakehouse Infrastructure

组成的本地开发环境中，提供：

- 可完全离线恢复的基础设施供应链；
- 极速、确定性的环境拉起能力；
- 严格声明式的稳态治理模型；
- 可预测的控制面恢复能力；
- 面向开发者的低认知负荷抽象；
- 面向 AI Agent 的显式权限边界与安全操作域。

Atlas 不将自身定位为若干云原生工具的集合，而将平台视为一个由多个独立 Reconciler 共同组成的分层控制系统。

平台设计遵循 CNCF Platform Engineering 的核心思想：

Platform as a Product Self-Service Declarative Automation Cognitive Load Reduction Clear Ownership Failure Domain Isolation

同时，Atlas 将 AI Agent 视为未来平台工程的一等参与者，因此物理目录、控制权、信任级别和执行权限必须能够被机器明确判断，而不能依赖隐含的人类经验。

0.3 2. 核心设计原则

Atlas 建立在以下不可妥协的架构原则之上。

0.3.1 2.1 Bootstrap ≠ GitOps

__INLINE_CODE_0__ 与 __INLINE_CODE_1__ 属于完全不同的生命周期域。

Bootstrap Bootstrap 是一次性、命令式、可退出的控制面建立过程。

其职责仅包括：

- 宿主环境检查；
- Kubernetes substrate 建立；
- Local Artifact Registry 建立与预热；
- 极简 Argo CD Seed 安装；
- Tier-0 Bootstrap AppProject 建立；
- External Root Anchor 实例化。

Bootstrap MUST NOT 长期参与平台稳态资源管理。

完成 GitOps 控制权移交后：

Bootstrap MUST terminate as an active control authority.

GitOps __INLINE_CODE_0__ 保存平台长期 Desired State。
Argo CD 负责持续比较：

Git Desired State
↓
Kubernetes Actual State

并持续进行收敛。

Bootstrap 与 GitOps 的关系不是双向共同管理，而是：

Bootstrap
|
| instantiate
▼
GitOps Control Plane
|
| adopt
▼
Steady State

控制权转移 MUST 是单向的。

0.3.2 2.2 四阶控制权模型

Atlas 将系统控制权划分为四个不同的 Authority Domain。

Git owns Definition Git 是平台期望状态的定义权威。

Git 保存：

- Root Anchor 契约模板；
- Platform Desired State；
- Workload Desired State；
- Policy；
- Environment Overlay；
- Version Lock；
- Governance Intent。

Git 回答：

系统“应该是什么”。

Bootstrap owns Instantiation Bootstrap 拥有初始实例化权。

它负责：

- 读取环境上下文；
- 建立最小 Kubernetes / Argo Seed；
- 建立 Tier-0 Bootstrap Capability；
- 实例化 External Root Anchor。

Bootstrap 回答：

当 GitOps 本身尚不存在时，由谁启动 GitOps。

Argo CD owns GitOps Reconciliation Argo CD 是 Atlas GitOps Desired-State Domain 中唯一的持续同步权威。

它负责：

- Git 与 Kubernetes 收敛；
- Application 拓扑；
- Sync；
- Prune；
- Health Gate；
- Drift Correction。

Argo CD 不拥有应用内部运行时行为。

Operators own Runtime Behavior 资源进入 Kubernetes API 后，其运行时生命周期由 Kubernetes 原生 Controller 或 Domain Operator 负责。

例如：

```

Git
↓
Argo CD
↓
FlinkDeployment CR
↓
Flink Kubernetes Operator
↓
JobManager / TaskManager

```

或者：

```

Git
↓
Argo CD
↓
Redpanda Cluster CR
↓

```

Redpanda Operator

↓

Runtime Cluster

因此：

GitOps Reconciliation Authority $\neq$ Runtime Reconciliation Authority.

__INLINE_CODE_0__ 只是 Argo CD 对自身 Desired State 的自管理机制，不构成独立的第五控制域。

0.3.3 2.3 External Root Anchor + Two-Level Reconciliation DAG

Atlas 正式采用：

External Root Anchor + Two-Level Reconciliation DAG Pattern

作为 GitOps 拓扑的架构基线。
这是 Atlas v1.0.2 的核心拓扑不变量。

2.3.1 External Root Anchor External Root Anchor 是 Atlas GitOps 控制图的外部信任锚点。它：

- 定义于 Git；
- 由 Bootstrap 根据环境上下文实例化；
- 作为 Root App-of-Apps 的入口；
- 不由另一个 Argo CD Application 持续调和。

其目的是显式终止无限自引用：

Who manages Argo CD?

↓

argocd-self

Who manages argocd-self?

↓

Platform Control DAG

Who manages Platform Control?

↓

External Root Anchor

Who manages External Root Anchor?

↓

Git Definition

+

Bootstrap Instantiation

+

Human / Policy Governance

External Root Anchor MUST NOT 被设计为自我管理 Application。

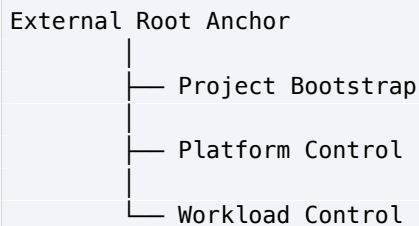
2.3.2 Level 1 —Root Macro DAG Root 层只表达：

Trust Transition 与 Macro Readiness Gate

Root MUST NOT 成为 Platform Component Catalog。
Root 不应知道：

- Flink；
- Envoy Gateway；
- Redpanda；
- MinIO；
- Loki；
- Tempo；
- Alloy；
- 未来新增的普通平台组件。

Root 只负责少量稳定的控制域跃迁：



因此普通平台能力增加或删除 SHOULD NOT 修改 __INLINE_CODE_0__。

2.3.3 Level 2 —Platform Capability DAG 平台组件的实际依赖关系由：

gitops/platform/applications/

表达。

Platform Control 负责：

- Foundation；
- Management；
- Operators；
- Infrastructure Controllers；
- Platform Services。

典型逻辑依赖为：

```
Foundation
  ↓
Management
  ↓
Operators
  ↓
Controllers
  ↓
Platform Services
```

Root 仅关心：

Platform Control Healthy?

而不关心其内部拥有多少组件。

2.3.4 Root manages trust transitions 正式约束：

Root SHALL model trust-domain transitions, not platform components.

任何只是因为新增：

- 数据库；
- Operator；
- Observability Backend；
- Gateway Controller；
- Messaging Engine；

而要求修改 Tier-0 Root 的设计，默认视为架构异味。

2.3.5 Leaf Application 是最小失败隔离单元 Atlas 将独立 Argo CD Application 视为最小的：

- Sync Unit；
- Health Unit；
- Autosync Unit；
- Recovery Unit；
- Failure Isolation Unit。

例如：

```
argocd-self
sealed-secrets
flink-operator
envoy-gateway
kube-prometheus-stack
redpanda
minio
```

SHOULD 分别成为独立 Leaf Application。

Application 不应仅因为文件系统存在子目录而进一步嵌套。

2.3.6 Application 嵌套深度限制 External Root Anchor 以下默认最多允许两个 Application orchestration hops:

```

External Root Anchor
    ↓
Control Application
    ↓
Leaf Application
    ↓
Kubernetes Resources / CRs
  
```

禁止形成:

```

Root
  ↓
Platform
  ↓
Domain
  ↓
Subdomain
  ↓
Component
  ↓
Resources
  
```

仅当存在真实的:

- Trust Boundary;
- Ownership Boundary;
- Failure Isolation Boundary;

时才允许增加新的 Application 层级。
突破默认深度 MUST 通过 ADR 明确批准。

0.3.4 2.4 Directory Ownership ≠ Dependency Ordering

Atlas 强制区分两个正交维度:

Directory 回答:

这个能力属于哪个领域?

Sync Wave 回答:

这个能力在什么依赖条件满足后才允许收敛?

因此:

```

management/
operators/
networking/
observability/
messaging/
storage/
  
```

表达领域归属。
而：

```
Management
Operators
Controllers
Platform Services
```

表达依赖阶段。
Atlas MUST NOT 创建：

```
controllers/
platform-services/
```

仅用于机械对应 Sync Wave。
同一个领域内允许存在不同依赖阶段。
例如：

```
networking/
├─ envoy-gateway-controller
└─ gateway-runtime-resources
```

可以分别处于不同 Wave。

0.3.5 2.5 GitOps Reconciliation Authority

Argo CD 是 Atlas GitOps Desired-State Domain 的唯一持续同步权威。
平台禁止同时引入第二个与 Argo CD 对相同资源进行 Desired-State Reconciliation 的控制器。
例如禁止：

```
Argo CD
+
Helm Release State
+
另一个 GitOps Controller
```

共同竞争同一资源所有权。
Helm 在 Atlas 中仅作为：

```
Template Renderer
```

使用。
Atlas MUST NOT 依赖 Helm Release History 作为平台稳态的一部分。

0.3.6 2.6 Artifacts ≠ Intent

Atlas 将软件发行产物与平台治理意图彻底分离。

Artifacts 第三方发行制品属于 Supply Chain Artifact。
例如：

- Helm Chart __INLINE_CODE_0__；
- OCI Image；
- CRD Bundle；

- 其他不可变发行资产。

复杂第三方 Helm Chart SHOULD 以不可变形式缓存，并在 Root DAG 启动前完成本地可用性准备。

Intent Atlas 自身的治理意图属于 Git Desired State，例如：

```
gitops/**/values.yaml
resources/
overlays/
policies/
```

Argo CD 在渲染阶段将：

```
Immutable Upstream Artifact
      +
Atlas Governance Intent
      ↓
Hydrated Desired State
```

进行组合。

2.6.1 Offline Supply Chain Invariant “完全离线”不只意味着 Helm Chart 本地可用。

Atlas 的离线不变量是：

Runtime 所需的全部外部 Artifact MUST 在断开公网前被本地验证并可用。

因此离线供应链最终必须同时覆盖：

```
Helm / OCI Artifacts
+
Container Images
+
Version Metadata
+
Required CRDs
```

具体缓存、镜像同步和 Registry Distribution 机制由独立 Supply Chain Design 或 RFC 管辖。

0.3.7 2.7 Developer Cognitive Load Reduction

Atlas 根据使用者角色分级暴露复杂性。

Platform Infrastructure 平台基础设施允许使用 Helm。

原因是：

- 上游生态复杂；
- 平台团队拥有治理复杂性的能力；
- 应最大化复用成熟发行资产。

Developer Workloads 普通业务工作负载优先使用：

- Plain YAML；
- Kustomize。

避免将 Helm Template Complexity 暴露给普通开发者。

Future Platform API 长期演进方向为：

```
apiVersion: atlas.io/v1alpha1
kind: StreamingApplication
```

由平台 Operator 将高层领域模型展开成：

- Flink；
- Kafka / Redpanda；
- Gateway；
- Observability；
- Policy；

等底层资源。

0.4 3. 标准目录与物理信任边界

Atlas 的目录结构同时表达：

- Domain Ownership；
- Trust Boundary；
- Agent Permission Boundary。

标准结构如下：

```
atlas/
├── env/                                # Host Injection Context
├── bootstrap/                          # Phase 0 / Imperative Executor
│   ├── argocd/
│   │   ├── install.sh
│   │   ├── atlas-bootstrap-project.yaml
│   │   └── root-app.yaml.tpl
├── gitops/                             # Phase 1+ / Desired State
│   ├── root/                          # □ Tier-0 Trust Boundary
│   │   ├── base/
│   │   │   ├── kustomization.yaml
│   │   │   ├── project-bootstrap-app.yaml
│   │   │   ├── platform-control-app.yaml
│   │   │   └── workload-control-app.yaml
│   └── overlays/
```



0.4.1 3.1 __INLINE_CODE_0__

__INLINE_CODE_0__ 属于 Tier-0 Architecture Trust Boundary。

AI Agent 可以：

- READ;
- ANALYZE;
- PROPOSE;
- GENERATE。

AI Agent MUST NOT 在无人判断的情况下：

- MERGE;

- APPLY;
- 修改 Root Trust Chain;
- 修改 Tier-0 AppProject Capability。

Tier-0 的自主修改必须经过 Human Judgment Gate。

0.4.2 3.2 __INLINE_CODE_0__

该目录不是新的 Platform Domain。
它只表达：

Platform Application Graph / Reconciliation Metadata.

其中只应保存：

- Argo CD Application;
- Kustomization;
- Environment Overlay;
- 与 DAG 本身直接相关的调和元数据。

实际组件 Desired State MUST 位于其领域目录。
例如：

platform/applications/base/envoy-gateway-app.yaml

只定义 Application。
实际组件资产仍位于：

platform/networking/envoy-gateway/

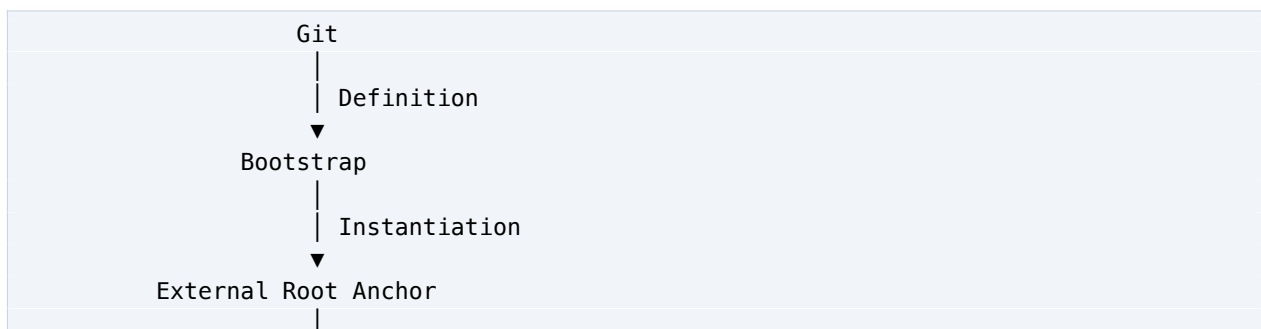
因此：

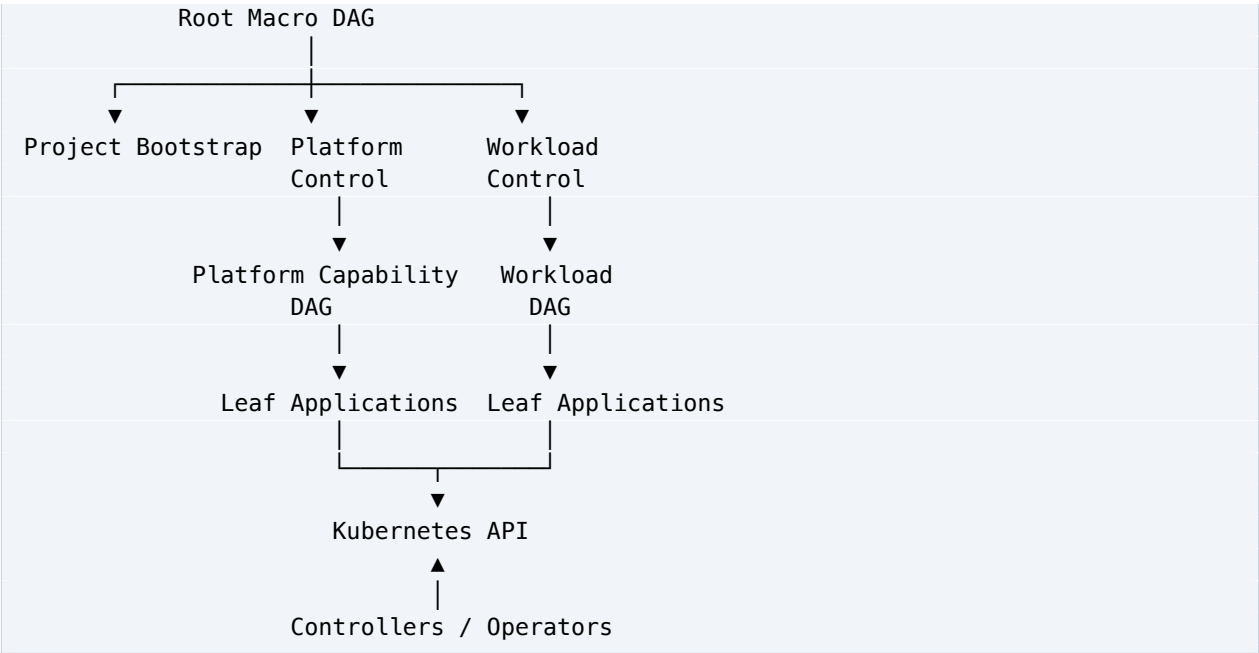
WHERE does it belong?
→ Domain Directory

WHEN may it converge?
→ Application DAG

0.5 4. Atlas 控制拓扑

Atlas 的逻辑控制图如下：





Root 与 Platform DAG 属于不同的同步作用域。
Sync Wave 不构成全平台共享的“绝对时间轴”。
其具体编号和 Health Gate 语义由：

Atlas GitOps Control Plane Design v1.0.3

规范。

0.6 5. 工程约束与防御机制

0.6.1 5.1 Dual Deletion Protection

Atlas 将两类完全不同的灾难模型分开处理。

Git-side Accidental Deletion 针对由 Parent Application 管理的 Tier-0 / Tier-1 Child Application CR：

```
argocd.argoproj.io/sync-options: Prune=confirm
```

Git 中误删除 Child Application 后：

```
Git deletion
  ↓
Parent detects prune
  ↓
Human confirmation required
```

Explicit Application Deletion Atlas 的核心 Tier-0 / Tier-1 Application 默认：

MUST NOT carry cascading resources-finalizer.

因此：

```

delete Application CR
    ↓
Application disappears
    ↓
Managed runtime resources survive

```

控制面事故不应自动演变为数据面销毁事故。

5.1.1 Protection Scope Dual Deletion Protection 主要保护的是：

Application Control Graph

它并不自动保护所有 descendant resources。
例如：

```

Redpanda Application
    ↓
StatefulSet / PVC / Cluster CR

```

如果 Leaf Application 对其资源执行 prune, Parent Application 上针对 Child Application CR 的 __INLINE_CODE_0__ 不会自动保护其内部 StatefulSet。

高价值资源必须根据领域需求额外采用：

- resource-level __INLINE_CODE_0__；
- __INLINE_CODE_0__；
- Admission Policy；
- Domain-native protection；
- Backup / Restore。

因此：

Control Graph Deletion Protection ≠ Managed Resource Deletion Protection.

0.6.2 5.2 Bootstrap Adoption Invariant

Bootstrap Seed 与 __INLINE_CODE_0__ MUST 满足无损接管契约：

```

Object Identity Parity
+
Version Parity
+
Health Capability Parity
+
CRD Compatibility
=
Bootstrap Adoption Contract

```

Object Identity Parity Namespace、核心对象身份和关键资源命名必须兼容。

Version Parity Bootstrap 与 __INLINE_CODE_0__ MUST 使用同一份：

```
versions.lock
```

作为版本权威。

Health Capability Parity Seed 与 Full Desired State MUST 注入相同的 Application Health Capability。

CRD Compatibility 恢复 Seed 不得造成：

- CRD downgrade；
- Schema incompatibility；
- Controller binary / CRD mismatch。

Bootstrap 的目标不是：

安装一个“能跑”的 Argo CD。

而是：

实例化一个能够被 Git Desired State 无损接管的最小控制面。

0.6.3 5.3 Zero-Plaintext Secret

任何未加密的 Secret，包括 Base64 编码内容：

MUST NOT enter Git.

Phase 1 采用 Sealed Secrets：

```
kubeseal
  ↓
SealedSecret
  ↓
Git
```

Controller Private Key 属于 Trust Root，必须物理隔离备份。

同时产生如下强依赖：

```
Sealed Secrets Controller
  ↓ Healthy
SealedSecret Materialization
  ↓
Secret Consumers
```

任何 Secret Consumer 不得先于 Secret Materialization Capability Ready。

Phase 2A 演进至：

External Secrets Operator
+
External Secret Manager

例如：

- Vault；
- AWS Secrets Manager；
- GCP Secret Manager；
- 等价外部密钥系统。

0.6.4 5.4 Tier-0 Human Judgment

Atlas 不允许 AI Agent 在平台最核心的信任根拥有完全自治权。

以下操作 MUST 经过 Human Judgment：

- Root Trust Chain 修改；
- Tier-0 AppProject 权限扩张；
- Break-Glass Recovery；
- Destructive Data Migration；
- Secret Trust Root 操作；
- 其他能够跨越信任域的高爆炸半径操作。

AI-Native 并不意味着：

Agent controls everything.

Atlas 的目标是：

Agent has maximum autonomy inside explicitly bounded safe domains.

0.7 6. Unified Telemetry & Observability

Atlas 将统一观测能力视为平台基础能力，而非应用上线后的附加功能。

0.7.1 6.1 Metrics

采用：

- Kube-Prometheus-Stack；
- Prometheus；
- OpenMetrics。

覆盖：

- Kubernetes Control Plane；
 - Platform Components；
 - Operators；
 - Application Services。
-

0.7.2 6.2 Collection

统一采集代理采用：

- Grafana Alloy；
 - OpenTelemetry Collector。
-

0.7.3 6.3 Logs

采用 Loki 作为日志后端，解决：

- Pod 重建；
- Node 漂移；
- Container 生命周期结束；

后的历史追溯问题。

0.7.4 6.4 Traces

采用 Tempo 作为 Trace Backend。
优先使用：

OTLP

作为跨服务遥测传输协议。

包括：

- Envoy Gateway；
 - Platform Services；
 - Streaming Workloads；
 - Microservices。
-

0.8 7. Disaster Recovery Boundaries

Atlas 严格区分：

Infrastructure Recreation 与 Stateful Business Recovery

0.8.1 7.1 Disposable State

以下状态默认可丢弃：

- Stateless Pod；
- Controller Runtime Cache；
- Git 可重建的 Kubernetes Objects；
- 临时执行环境；
- Bootstrap `__INLINE_CODE_0__`。

这些对象应通过 GitOps 或 Controller 快速重建。

0.8.2 7.2 Must-Backup State

以下资产必须拥有独立备份方案：

Git Repository Git 是 Desired State 唯一权威。

Cluster Trust Roots 包括：

- Sealed Secrets Private Key；
- TLS Root；
- 外部密钥系统恢复凭证；
- 其他平台 Trust Material。

Business Data 包括：

- Kafka / Redpanda Logs；
- Database Physical Data；
- MinIO Object Data；
- Stateful Application Data。

0.8.3 7.3 Application-consistent Recovery

Atlas 明确规定：

Storage-level backup MUST NOT be treated as application-consistent recovery.

例如：

Flink 依赖：

- Checkpoint；
- Savepoint。

Kafka / Redpanda 依赖：

- Broker Replication；
- Cluster Metadata；
- Operator-aware Recovery。

Database 依赖：

- Database-native Backup / Restore；
- WAL / Binlog / Snapshot Semantics。

Object Storage 依赖：

- Object-level consistency；
- Replication；
- Versioning；
- Domain-native recovery mechanism。

PVC Snapshot 只能作为底层能力，不得替代领域恢复语义。

0.9 8. Break-Glass Architecture Principle

Atlas 在灾难情况下执行：

Freeze Outside-In, Recover Inside-Out.

0.9.1 Freeze

```
External Root
  ↓ freeze
Child Control Applications
  ↓ freeze
Affected Reconciliation Graph
```

优先阻断错误传播。

0.9.2 Recover

```
Seed
↓
argocd-self
↓
Platform Capability DAG
↓
Workload Control
↓
External Root Resume
```

优先恢复最内层可信控制能力，再逐级恢复外层调和。

具体 Break-Glass Procedure 由 GitOps Control Plane Design 管辖。

0.10 9. Architecture Invariants

Atlas v1.0.2 正式冻结以下平台级不变量。

0.10.1 INV-01 —Definition Authority

Git MUST remain the authoritative definition of Desired State.

0.10.2 INV-02 —Bootstrap Termination

Bootstrap MUST cease active control after GitOps adoption.

0.10.3 INV-03 —External Root

External Root Anchor MUST remain outside parent Application reconciliation.

0.10.4 INV-04 —GitOps Authority

Argo CD MUST remain the sole continuous GitOps Desired-State reconciler.

0.10.5 INV-05 —Runtime Authority

Runtime lifecycle MUST belong to Kubernetes Controllers or Domain Operators.

0.10.6 INV-06 —Two-Level DAG

Atlas MUST use a minimal Root Macro DAG plus delegated capability-level DAGs.

0.10.7 INV-07 —Root Minimalism

Ordinary platform component lifecycle changes MUST NOT require Tier-0 Root changes.

0.10.8 INV-08 —Domain / Dependency Separation

Directory structure MUST represent domain ownership; dependency ordering MUST be represented separately.

0.10.9 INV-09 —Leaf Failure Isolation

Independent platform capabilities SHOULD map to independent Leaf Applications.

0.10.10 INV-10 —Bounded Application Depth

Additional App-of-Apps nesting MUST NOT be introduced solely to mirror directory hierarchy.

0.10.11 INV-11 —Adoption Parity

Bootstrap Seed and argocd-self MUST satisfy the Bootstrap Adoption Contract.

0.10.12 INV-12 —Control/Data Failure Isolation

Deleting or damaging the GitOps control graph MUST NOT implicitly destroy high-value runtime/data-plane state.

0.10.13 INV-13 —Zero Plaintext Secret

Unencrypted Secret material MUST NOT enter Git.

0.10.14 INV-14 —Offline Artifact Availability

All runtime-critical external artifacts MUST be locally available before entering offline operation.

0.10.15 INV-15 —Stateful Recovery Semantics

Application-consistent recovery MUST be owned by the corresponding stateful domain.

0.10.16 INV-16 —Tier-0 Human Judgment

Trust-boundary-changing operations MUST remain human-gated unless explicitly authorized by a future governance standard.

0.11 10. Conformance Model

任何 Atlas 实现只有在同时满足：

```
Architecture Invariants
+
GitOps Domain Contracts
+
Repository Structural Rules
+
Runtime Policy Controls
```

时，才可以声明：

Atlas Architecture Conformant.

后续仓库审计应以：

INV-*

MUST

MUST NOT

SHOULD

为机械审查基准，而不是依赖自然语言主观解释。

0.12 11. Architecture Freeze Statement

Atlas Architecture Design v1.0.2 正式冻结以下核心架构：

External Root Anchor + Two-Level Reconciliation DAG

其中：

- Root 只管理 Trust Transition；
- Platform Control 管理 Capability Dependency；
- Workload Control 管理 Workload Orchestration；
- Leaf Application 构成最小 Failure Isolation Unit；
- Domain Directory 与 Dependency Wave 完全解耦；
- Bootstrap、Argo CD、Operators 保持严格控制权边界。

未来普通平台组件的增加、删除与升级：

SHOULD NOT require modification of the Tier-0 Root Architecture.

任何破坏上述模式的变更均视为 Architecture Change，必须通过正式 ADR / Architecture Review，而不得作为普通实现重构直接合入。